Computational Economics

Lecture: Q Functions, Optimality, and Value Function Iteration

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lecture Overview

- **From State Values to Action Values**
  - Evaluate a maze policy, then compare first actions
  - Define Q^σ and link it to V^σ
- **The Principle of Optimality**
  - Improve a continuation from a reached state
  - Derive optimal state and action values
- **Value Function Iteration**
  - Update values, check convergence, and recover a policy
  - Apply the algorithm to job search and continuous saving

Recap: The Value Function

For a given deterministic policy σ,

$$V^{\sigma}(s) = \underbrace{r\big(s, \sigma(s)\big)}_{\text{Immediate reward}} + \underbrace{\beta\, \mathbb{E}\big[V^{\sigma}(s') \mid s,\ a = \sigma(s)\big]}_{\text{Discounted continuation value}}.$$

s' is the next state, drawn from $P(\cdot \mid s, \sigma(s))$.

Recap: Computing the Value Function of a Given Policy

Consider a specific deterministic policy σ.

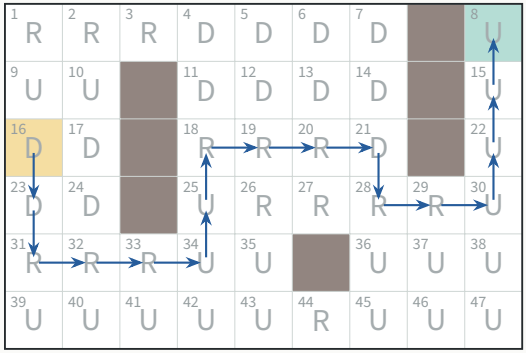


Figure 1: Actions and path under σ.

- U: up; D: down;
  L: left; R: right.
- Each move gives -1, including entry into cell 8.
- Set $\beta = 1$ for simplicity.
- Following this policy reaches cell 8 from every free cell.

Recap: Computing the Value Function of a Given Policy

With $\beta = 1$, compute values backward from cell 8, following the actions of σ.

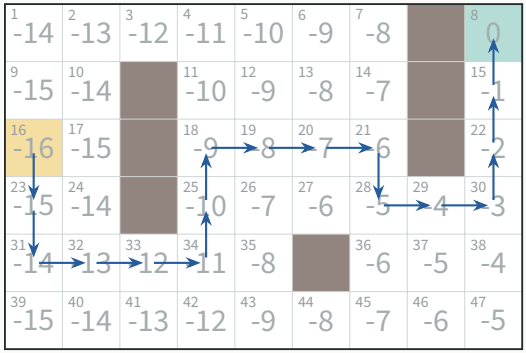


Figure 2: Values and path under σ.

$V^\sigma(8) = 0$. For every other cell,

$$V^\sigma(s) = -1 + V^\sigma(s'),$$

where s' is its chosen next cell.

$$V^\sigma(15) = -1 + 0 = -1,$$
$$V^\sigma(22) = -1 + (-1) = -2,$$
$$V^\sigma(30) = -1 + (-2) = -3.$$

Continuing backward gives $V^\sigma(16) = -16$.

Each value is minus the number of moves to cell 8 under σ.

Example: Comparing First Actions

At cell 25, the displayed policy chooses U. Compare U with R, then follow σ from the next cell. Keep $\beta = 1$.

- **Up:** move to cell 18, whose value is $V^\sigma(18) = -9$:

$$-1 + V^\sigma(18) = -10.$$

- **Right:** move to cell 26, whose value is $V^\sigma(26) = -7$:

$$-1 + V^\sigma(26) = -8.$$

Both actions cost one now. Right gives a better continuation.

$V^\sigma(25) = -10$ evaluates the policy's choice. An **action value** evaluates each possible first choice separately.

The Action-Value Function

The **action-value function** $Q^\sigma(s, a)$ is the expected return from state s, taking action a first and following σ afterward:

$$Q^\sigma(s, a) = \mathbb{E}_\sigma[G_t \mid s_t = s,\ a_t = a], \qquad G_t = \sum_{\tau=0}^{\infty} \beta^\tau r(s_{t+\tau}, a_{t+\tau}).$$

- The first action is deliberately chosen; it need not be an action that σ selects at s.
- The expectation covers transitions and later actions under σ.
- Averaging returns from repeated rollouts starting with (s, a) provides an estimate of $Q^\sigma(s, a)$.

For the general discounted setting, assume bounded rewards, $0 < \beta < 1$, time-homogeneous dynamics, and a stationary policy. The terminating maze permits $\beta = 1$.

Linking Q and V

Fix the first action; evaluate the remaining rollout by its state value:

$$Q^{\sigma}(s,a) = r(s,a) + \beta\,\mathbb{E}\big[V^{\sigma}(s') \mid s,a\big]\,.$$

For a finite action set, average the first action according to the policy:

$$V^{\sigma}(s) = \sum_{a\in\mathcal{A}(s)} \Pr\{\sigma(s)=a\}\,Q^{\sigma}(s,a).$$

For a deterministic policy,

$$V^{\sigma}(s) = Q^{\sigma}(s,\sigma(s)).$$

- V^{σ} evaluates a state; Q^{σ} distinguishes actions at that state.
- Once Q^{σ} is known, compare actions directly. The best first action maximizes $Q^{\sigma}(s,a)$, with σ governing the continuation.

Example: Improving the Continuation

Let $\widetilde{\sigma}$ change the action at cell 25 from U to R. All other actions stay the same.

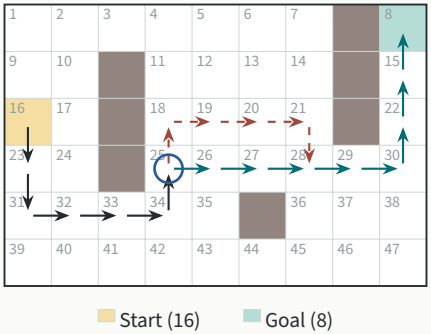


Figure 3: A common prefix and two continuations.

- Solid continuation: 8 moves.
- Dashed detour: 10 moves, including the shared final segment.

With $\beta = 1$,

$$V^{\sigma}(25) = -10,$$
$$V^{\widetilde{\sigma}}(25) = -8.$$

Keeping the six-move prefix gives

$$V^{\widetilde{\sigma}}(16) = -6 - 8 = -14.$$

The Principle of Optimality

In a finite MDP, an optimal policy must have an optimal continuation from every state reached with positive probability.

1. Suppose a better continuation is available at a reached state.
2. Keep the earlier decisions unchanged and use the better continuation there.
3. The expected return increases, contradicting the original policy's optimality.

This argument uses the **Markov state**: it contains the information needed to evaluate future choices.

With uncertain transitions, choose a continuation for each possible next state. Optimality concerns expected return across those outcomes.

Optimal State and Action Values

For a finite discounted MDP, an optimal stationary deterministic policy exists. Define

$$V^*(s) = \max_\sigma V^\sigma(s), \qquad Q^*(s,a) = \max_\sigma Q^\sigma(s,a).$$

The principle of optimality gives

$$\begin{aligned} Q^*(s,a) &= r(s,a) + \beta\, \mathbb{E}\left[V^*(s') \mid s, a\right], \\ V^*(s) &= \max_{a \in \mathcal{A}(s)} Q^*(s,a). \end{aligned}$$

- Q^*: fix the current action, then choose optimally afterward.
- V^*: also choose the current action optimally.
- The expectation averages the outcomes of the chosen action; it does not let us choose the random outcome.

Optimality as a One-Step Look Ahead

$$V^*(s) = \max_a Q^*(s, a)$$

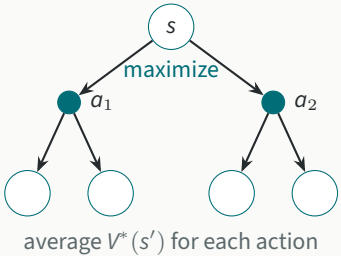


Figure 4: Choose now, average outcomes.

$$Q^*(s, a) = r(s, a) + \beta \mathbb{E}[\max_{a'} Q^*(s', a') \mid s, a]$$

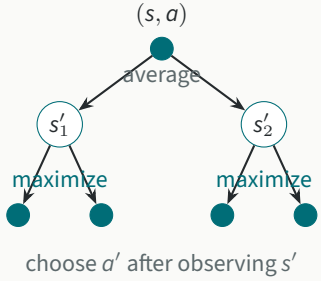


Figure 5: Fix now, choose at the next state.

Open circles: states. Filled circles: state–action pairs. Adapted from Sutton and Barto (2018), Figure 3.4; branches show possible next states.

The Bellman Optimality Operator

Replace the unknown optimal continuation value by a candidate function v. The **Bellman optimality operator** produces

$$(Tv)(s) = \max_{a \in \mathcal{A}(s)} \left\{ r(s,a) + \beta \sum_{s'} P(s' \mid s,a) v(s') \right\}.$$

- For each action, add the immediate reward to the expected discounted continuation value; take the largest result.
- The optimal value is a **fixed point**: $TV^* = V^*$.

The sum describes discrete next states; use a probability integral for continuous next states. Unlike fixed-policy evaluation, T maximizes over the current action.

Value Function Iteration

Given a finite MDP, its reward function and transition kernel, and value-error tolerance $\varepsilon > 0$:

1. Initialize $v_0(s) = 0$ and set $k = 0$. Keep zero-reward absorbing goal states at zero.
2. Score every feasible action using the **old** value function:

$$Q_k(s, a) = r(s, a) + \beta \, \mathbb{E}[v_k(s') \mid s, a] \, .$$

3. Store the best score in a new array: $v_{k+1}(s) = \max_a Q_k(s, a)$.
4. Compute $d_k = \max_s |v_{k+1}(s) - v_k(s)|$. If $d_k \leq (1 - \beta)\varepsilon$, return v_k; otherwise increase k and repeat.

Q_k uses the approximate continuation v_k; it need not equal Q^σ for a particular policy.

A **sweep** updates every state once. The separate new array ensures all updates use the same v_k.

Example: Values Move Back Through the Maze

Consider cells 30, 22, and 15 near the goal (8) in the same maze. Set $\beta = 0.9$, $v_0 = 0$; each move before the goal costs one.

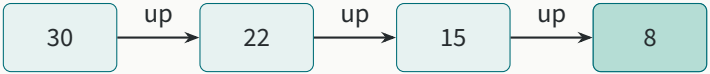


Figure 6: The final corridor to the goal.

	Cell 30	Cell 22	Cell 15	Cell 8
v_0	0	0	0	0
v_1	-1	-1	-1	0
v_2	-1.9	-1.9	-1	0
v_3	-2.71	-1.9	-1	0

For example, $v_3(30) = -1 + 0.9v_2(22) = -2.71$. For this zero-initialized update, v_k is the optimal k-period return with zero value afterward.

Why VFI Converges

For finite states, use $\|v - w\|_\infty = \max_s |v(s) - w(s)|$. The difference between two scores for the **same action** satisfies

$$|\beta \mathbb{E}[v(s') - w(s') \mid s, a]| \leq \beta \|v - w\|_\infty.$$

Since $|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|$,

$$\|Tv - Tw\|_\infty \leq \beta \|v - w\|_\infty.$$

Thus T is a **contraction**: the new maximum distance is at most β times the old distance.

- There is a unique fixed point V^*; every initial finite vector converges to it.
- After k sweeps,
$$\|v_k - V^*\|_\infty \leq \beta^k \|v_0 - V^*\|_\infty.$$
- A discount factor closer to one gives a slower worst-case rate.

A Stopping Rule With an Error Bound

After computing $v_{k+1} = Tv_k$, measure the **Bellman residual** of v_k:

$$d_k = \|Tv_k - v_k\|_\infty = \|v_{k+1} - v_k\|_\infty.$$

The contraction inequality gives

$$\begin{aligned}\|v_k - V^*\|_\infty &\le \|v_k - Tv_k\|_\infty + \|Tv_k - TV^*\|_\infty \\ &\le d_k + \beta\|v_k - V^*\|_\infty,\end{aligned}$$

so $\|v_k - V^*\|_\infty \le d_k/(1-\beta)$.

- To guarantee value error at most ε, stop when $d_k \le (1-\beta)\varepsilon$ and return v_k.
- At $\beta = 0.9$ and $\varepsilon = 0.01$, require $d_k \le 0.001$.

This guarantee uses exact expectations and maximization. Numerical approximation introduces additional errors.

Recovering a Policy

Use the action scores calculated from the returned v_k. A **greedy policy** chooses

$$\widehat{\sigma}(s) \in \arg\max_{a \in \mathcal{A}(s)} Q_k(s, a).$$

- If $v_k = V^*$, then $Q_k = Q^*$, and every maximizing action is optimal.
- With approximate values, evaluate the resulting policy to check its achieved return.
- If several actions tie, choose one consistently.

VFI updates value estimates. The final maximization converts those estimates into a decision rule for each state.

Example: VFI for McCall Job Search

The state is (U, w) or (E, w). Rejecting pays b and draws a new offer w' with probability $q(w')$; accepting pays w forever.

$$v_k((\mathrm{E}, w)) = \frac{w}{1-\beta} \quad \text{for every } k.$$

Use this known employed value and update unemployed states:

$$\begin{aligned} Q_k((\mathrm{U}, w), \text{accept}) &= \frac{w}{1-\beta}, \\ Q_k((\mathrm{U}, w), \text{reject}) &= b + \beta \sum_{w'} q(w') v_k((\mathrm{U}, w')), \\ v_{k+1}((\mathrm{U}, w)) &= \max\{Q_k((\mathrm{U}, w), \text{accept}),\ Q_k((\mathrm{U}, w), \text{reject})\}. \end{aligned}$$

Initialize $v_0((\mathrm{U}, w)) = 0$. Employed-state values stay fixed at their known values; only unemployed-state values need iteration.

Example: McCall Iterations

Use $\beta = 0.9$, $b = 1$, offers $(1, 2, 3, 4, 5)$, and probabilities $(0.1, 0.2, 0.4, 0.2, 0.1)$.

	$w = 1$	$w = 2$	$w = 3$	$w = 4$	$w = 5$
v_0	0.00	0.00	0.00	0.00	0.00
v_1	10.00	20.00	30.00	40.00	50.00
v_2	28.00	28.00	30.00	40.00	50.00
v_3	31.06	31.06	31.06	40.00	50.00
v_{10}	34.20	34.20	34.20	40.00	50.00
V^*	34.32	34.32	34.32	40.00	50.00

The columns give $v_k((\mathrm{U}, w))$. After the first update, the probability-weighted mean is 30. For offer 2, the next update is

$$v_2((\mathrm{U}, 2)) = \max\{20,\ 1 + 0.9 \times 30\} = 28.$$

At convergence, waiting is worth about 34.32: reject offers 1–3 and accept 4–5.

Example: VFI for Continuous Saving

With storage capacity $M \geq 0$, stock is $s \in [0, M + B]$. Choose saving $a \in [0, \min(s, M)]$ and consume $c = s - a$.

- Income $Y' \sim \text{Uniform}[0, B]$, $B > 0$, is independent across periods; next stock is $s' = a + Y'$.
- Reward is $r(s, a) = (s - a)^\alpha$, with $0 < \alpha < 1$.

The update averages over income and maximizes over saving:

$$Q_k(s, a) = (s - a)^\alpha + \frac{\beta}{B} \int_0^B v_k(a + y)\, dy,$$
$$v_{k+1}(s) = \max_{a \in [0, \min(s, M)]} Q_k(s, a).$$

The exact operator is a contraction under the supremum norm on continuous value functions. The state interval is compact, rewards are bounded, and $0 < \beta < 1$.

Computing the Continuous Update

To implement the continuous saving model numerically:

1. **Represent v_k:** store values at stock points and **interpolate** between them.
2. **Compute the expectation:** approximate the income integral using **numerical quadrature** or sampled incomes.
3. **Choose saving:** maximize over the feasible interval using numerical optimization or an action grid.

These approximations add errors beyond the VFI stopping error. Refine the stock grid, integration, and maximization to check that values and decisions are stable.

The economic model's states and actions remain continuous; the grid is a computational representation.

References and Further Reading

1. **Richard S. Sutton and Andrew G. Barto (2018), Reinforcement Learning: An Introduction. 2nd ed. MIT Press. [Book]**
   - Section 3.5: Policies and Value Functions
   - Section 3.6: Optimal Policies and Optimal Value Functions
   - Section 4.4: Value Iteration

2. **Thomas J. Sargent and John Stachurski, QuantEcon. [Online lectures]**
   - Discrete State Dynamic Programming — The Bellman Equation and the Principle of Optimality
   - Discrete State Dynamic Programming — Value Function Iteration
   - Job Search I: The McCall Search Model — The McCall Model
   - Discrete State Dynamic Programming — Example: A Growth Model
   - The Income Fluctuation Problem I: Basic Model — The Optimal Savings Problem